# An introduction to the ipaddress module

***Release 3.5.2***

**Guido van Rossum**
**and the Python development team**

**June 25, 2016**

**Python Software Foundation**
**Email: docs@python.org**

## Contents

**1 Creating Address/Network/Interface objects** **1**
- 1.1 A Note on IP Versions . . . 2
- 1.2 IP Host Addresses . . . 2
- 1.3 Defining Networks . . . 2
- 1.4 Host Interfaces . . . 3

**2 Inspecting Address/Network/Interface Objects** **3**

**3 Networks as lists of Addresses** **4**

**4 Comparisons** **5**

**5 Using IP Addresses with other modules** **5**

**6 Getting more detail when instance creation fails** **5**

**author** Peter Moody

**author** Nick Coghlan

**Overview**

This document aims to provide a gentle introduction to the `ipaddress` module. It is aimed primarily at users that aren't already familiar with IP networking terminology, but may also be useful to network engineers wanting an overview of how `ipaddress` represents IP network addressing concepts.

## Creating Address/Network/Interface objects

Since `ipaddress` is a module for inspecting and manipulating IP addresses, the first thing you'll want to do is create some objects. You can use `ipaddress` to create objects from strings and integers.

## A Note on IP Versions

For readers that aren't particularly familiar with IP addressing, it's important to know that the Internet Protocol is currently in the process of moving from version 4 of the protocol to version 6. This transition is occurring largely because version 4 of the protocol doesn't provide enough addresses to handle the needs of the whole world, especially given the increasing number of devices with direct connections to the internet.

Explaining the details of the differences between the two versions of the protocol is beyond the scope of this introduction, but readers need to at least be aware that these two versions exist, and it will sometimes be necessary to force the use of one version or the other.

## IP Host Addresses

Addresses, often referred to as "host addresses" are the most basic unit when working with IP addressing. The simplest way to create addresses is to use the `ipaddress.ip_address()` factory function, which automatically determines whether to create an IPv4 or IPv6 address based on the passed in value:

```
>>> ipaddress.ip_address('192.0.2.1')
IPv4Address('192.0.2.1')
>>> ipaddress.ip_address('2001:DB8::1')
IPv6Address('2001:db8::1')
```

Addresses can also be created directly from integers. Values that will fit within 32 bits are assumed to be IPv4 addresses:

```
>>> ipaddress.ip_address(3221225985)
IPv4Address('192.0.2.1')
>>> ipaddress.ip_address(42540766411282592856903984951653826561)
IPv6Address('2001:db8::1')
```

To force the use of IPv4 or IPv6 addresses, the relevant classes can be invoked directly. This is particularly useful to force creation of IPv6 addresses for small integers:

```
>>> ipaddress.ip_address(1)
IPv4Address('0.0.0.1')
>>> ipaddress.IPv4Address(1)
IPv4Address('0.0.0.1')
>>> ipaddress.IPv6Address(1)
IPv6Address('::1')
```

## Defining Networks

Host addresses are usually grouped together into IP networks, so `ipaddress` provides a way to create, inspect and manipulate network definitions. IP network objects are constructed from strings that define the range of host addresses that are part of that network. The simplest form for that information is a "network address/network prefix" pair, where the prefix defines the number of leading bits that are compared to determine whether or not an address is part of the network and the network address defines the expected value of those bits.

As for addresses, a factory function is provided that determines the correct IP version automatically:

```
>>> ipaddress.ip_network('192.0.2.0/24')
IPv4Network('192.0.2.0/24')
>>> ipaddress.ip_network('2001:db8::0/96')
IPv6Network('2001:db8::/96')
```

Network objects cannot have any host bits set. The practical effect of this is that `192.0.2.1/24` does not describe a network. Such definitions are referred to as interface objects since the ip-on-a-network notation is commonly used to describe network interfaces of a computer on a given network and are described further in the next section.

By default, attempting to create a network object with host bits set will result in `ValueError` being raised. To request that the additional bits instead be coerced to zero, the flag `strict=False` can be passed to the constructor:

```
>>> ipaddress.ip_network('192.0.2.1/24')
Traceback (most recent call last):
  ...
ValueError: 192.0.2.1/24 has host bits set
>>> ipaddress.ip_network('192.0.2.1/24', strict=False)
IPv4Network('192.0.2.0/24')
```

While the string form offers significantly more flexibility, networks can also be defined with integers, just like host addresses. In this case, the network is considered to contain only the single address identified by the integer, so the network prefix includes the entire network address:

```
>>> ipaddress.ip_network(3221225984)
IPv4Network('192.0.2.0/32')
>>> ipaddress.ip_network(42540766411282592856903984951653826560)
IPv6Network('2001:db8::/128')
```

As with addresses, creation of a particular kind of network can be forced by calling the class constructor directly instead of using the factory function.

### Host Interfaces

As mentioned just above, if you need to describe an address on a particular network, neither the address nor the network classes are sufficient. Notation like `192.0.2.1/24` is commonly used by network engineers and the people who write tools for firewalls and routers as shorthand for "the host `192.0.2.1` on the network `192.0.2.0/24`", Accordingly, `ipaddress` provides a set of hybrid classes that associate an address with a particular network. The interface for creation is identical to that for defining network objects, except that the address portion isn't constrained to being a network address.

```
>>> ipaddress.ip_interface('192.0.2.1/24')
IPv4Interface('192.0.2.1/24')
>>> ipaddress.ip_interface('2001:db8::1/96')
IPv6Interface('2001:db8::1/96')
```

Integer inputs are accepted (as with networks), and use of a particular IP version can be forced by calling the relevant constructor directly.

## Inspecting Address/Network/Interface Objects

You've gone to the trouble of creating an IPv(4|6)(Address|Network|Interface) object, so you probably want to get information about it. `ipaddress` tries to make doing this easy and intuitive.

Extracting the IP version:

```
>>> addr4 = ipaddress.ip_address('192.0.2.1')
>>> addr6 = ipaddress.ip_address('2001:db8::1')
>>> addr6.version
6
>>> addr4.version
4
```

Obtaining the network from an interface:

```
>>> host4 = ipaddress.ip_interface('192.0.2.1/24')
>>> host4.network
IPv4Network('192.0.2.0/24')
>>> host6 = ipaddress.ip_interface('2001:db8::1/96')
```

```
>>> host6.network
IPv6Network('2001:db8::/96')
```

Finding out how many individual addresses are in a network:

```
>>> net4 = ipaddress.ip_network('192.0.2.0/24')
>>> net4.num_addresses
256
>>> net6 = ipaddress.ip_network('2001:db8::0/96')
>>> net6.num_addresses
4294967296
```

Iterating through the "usable" addresses on a network:

```
>>> net4 = ipaddress.ip_network('192.0.2.0/24')
>>> for x in net4.hosts():
...     print(x)
192.0.2.1
192.0.2.2
192.0.2.3
192.0.2.4
...
192.0.2.252
192.0.2.253
192.0.2.254
```

Obtaining the netmask (i.e. set bits corresponding to the network prefix) or the hostmask (any bits that are not part of the netmask):

```
>>> net4 = ipaddress.ip_network('192.0.2.0/24')
>>> net4.netmask
IPv4Address('255.255.255.0')
>>> net4.hostmask
IPv4Address('0.0.0.255')
>>> net6 = ipaddress.ip_network('2001:db8::0/96')
>>> net6.netmask
IPv6Address('ffff:ffff:ffff:ffff:ffff:ffff::')
>>> net6.hostmask
IPv6Address('::ffff:ffff')
```

Exploding or compressing the address:

```
>>> addr6.exploded
'2001:0db8:0000:0000:0000:0000:0000:0001'
>>> addr6.compressed
'2001:db8::1'
>>> net6.exploded
'2001:0db8:0000:0000:0000:0000:0000:0000/96'
>>> net6.compressed
'2001:db8::/96'
```

While IPv4 doesn't support explosion or compression, the associated objects still provide the relevant properties so that version neutral code can easily ensure the most concise or most verbose form is used for IPv6 addresses while still correctly handling IPv4 addresses.

## Networks as lists of Addresses

It's sometimes useful to treat networks as lists. This means it is possible to index them like this:

```
>>> net4[1]
IPv4Address('192.0.2.1')
```

```
>>> net4[-1]
IPv4Address('192.0.2.255')
>>> net6[1]
IPv6Address('2001:db8::1')
>>> net6[-1]
IPv6Address('2001:db8::ffff:ffff')
```

It also means that network objects lend themselves to using the list membership test syntax like this:

```
if address in network:
    # do something
```

Containment testing is done efficiently based on the network prefix:

```
>>> addr4 = ipaddress.ip_address('192.0.2.1')
>>> addr4 in ipaddress.ip_network('192.0.2.0/24')
True
>>> addr4 in ipaddress.ip_network('192.0.3.0/24')
False
```

## Comparisons

`ipaddress` provides some simple, hopefully intuitive ways to compare objects, where it makes sense:

```
>>> ipaddress.ip_address('192.0.2.1') < ipaddress.ip_address('192.0.2.2')
True
```

A `TypeError` exception is raised if you try to compare objects of different versions or different types.

## Using IP Addresses with other modules

Other modules that use IP addresses (such as `socket` ) usually won't accept objects from this module directly. Instead, they must be coerced to an integer or string that the other module will accept:

```
>>> addr4 = ipaddress.ip_address('192.0.2.1')
>>> str(addr4)
'192.0.2.1'
>>> int(addr4)
3221225985
```

## Getting more detail when instance creation fails

When creating address/network/interface objects using the version-agnostic factory functions, any errors will be reported as `ValueError` with a generic error message that simply says the passed in value was not recognized as an object of that type. The lack of a specific error is because it's necessary to know whether the value is *supposed* to be IPv4 or IPv6 in order to provide more detail on why it has been rejected.

To support use cases where it is useful to have access to this additional detail, the individual class constructors actually raise the `ValueError` subclasses `ipaddress.AddressValueError` and `ipaddress.NetmaskValueError` to indicate exactly which part of the definition failed to parse correctly.

The error messages are significantly more detailed when using the class constructors directly. For example:

```
>>> ipaddress.ip_address("192.168.0.256")
Traceback (most recent call last):
  ...
ValueError: '192.168.0.256' does not appear to be an IPv4 or IPv6 address
>>> ipaddress.IPv4Address("192.168.0.256")
```

```
Traceback (most recent call last):
  ...
ipaddress.AddressValueError: Octet 256 (> 255) not permitted in '192.168.0.256'

>>> ipaddress.ip_network("192.168.0.1/64")
Traceback (most recent call last):
  ...
ValueError: '192.168.0.1/64' does not appear to be an IPv4 or IPv6 network
>>> ipaddress.IPv4Network("192.168.0.1/64")
Traceback (most recent call last):
  ...
ipaddress.NetmaskValueError: '64' is not a valid netmask
```

However, both of the module specific exceptions have `ValueError` as their parent class, so if you're not concerned with the particular type of error, you can still write code like the following:

```
try:
    network = ipaddress.IPv4Network(address)
except ValueError:
    print('address/netmask is invalid for IPv4:', address)
```